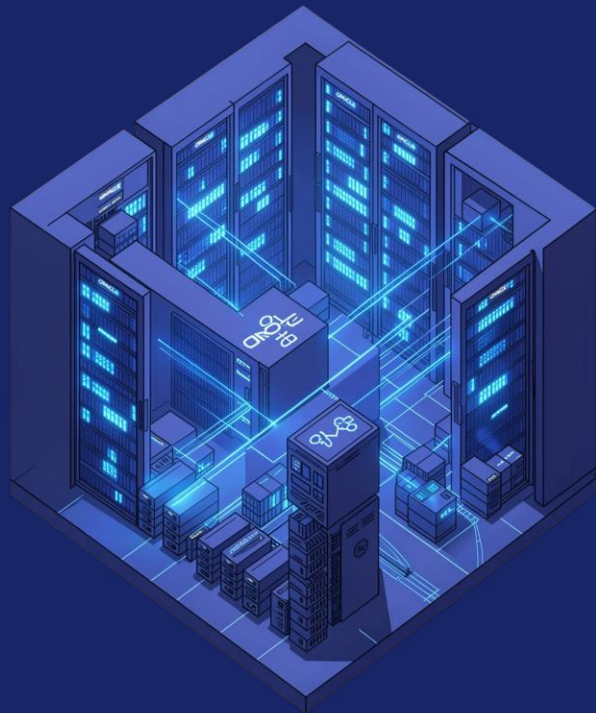




Riepilogo Completo SQL & PL/SQL

Da SELECT agli Oggetti PL/SQL

Questo materiale di ripasso è pensato per sviluppatori e analisti di livello base/intermedio che lavorano con **Oracle Database**. Il percorso copre l'intero spettro del linguaggio SQL — dalle query di base alle aggregazioni, dai JOIN al DML e DDL — e prosegue con PL/SQL, il linguaggio procedurale di Oracle: blocchi anonimi, variabili, gestione degli errori, cursori, procedure, function e package. Ogni argomento è accompagnato da esempi pratici basati sulle tabelle standard **EMPLOYEES**, **DEPARTMENTS** e **JOBS**.

ORACLE DATABASE

SQL

PL/SQL

LIVELLO BASE/INTERMEDIO

SELECT, Filtri, Ordinamento e Funzioni Base

Il database relazionale organizza i dati in **tabelle** composte da **righe** (record) e **colonne** (attributi). Le **chiavi primarie** identificano univocamente ogni riga; le **chiavi esterne** stabiliscono relazioni tra tabelle. In Oracle, il comando **SELECT** è il punto di partenza per estrarre dati.

Comandi Fondamentali

SELECT specifica le colonne da restituire. **FROM** indica la tabella sorgente. L'**alias** (AS) rinomina colonne o tabelle. È possibile includere **espressioni** calcolate e usare || per concatenare stringhe.

DISTINCT elimina i duplicati. **ORDER BY** ordina i risultati (ASC/DESC). La clausola **WHERE** filtra le righe prima dell'ordinamento.

Operatori di Filtro WHERE

- =, <>, >, < – Confronto diretto
- BETWEEN – Intervallo inclusivo
- IN – Lista di valori ammessi
- LIKE – Pattern con % e _
- IS NULL – Verifica valori nulli
- AND, OR, NOT – Connettori logici

Funzioni Base

- **Stringhe:** UPPER, LOWER, SUBSTR, INSTR
- **Numeri:** ROUND, TRUNC
- **Date:** SYSDATE, ADD_MONTHS

```
SELECT first_name || ' ' || last_name AS nome_completo,  
       salary * 12 AS stipendio_annuo  
FROM employees  
WHERE department_id IN (50, 80)  
       AND salary > 3000  
ORDER BY salary DESC;
```

 **Best Practice:** Evitare SELECT * in produzione. Specificare sempre le colonne necessarie per migliorare leggibilità e performance.

Funzioni Avanzate, Aggregazioni e GROUP BY

Le funzioni built-in di Oracle trasformano e analizzano i dati. Le **funzioni di aggregazione** (COUNT, SUM, AVG, MIN, MAX) calcolano valori su gruppi di righe. La clausola **GROUP BY** raggruppa i risultati; **HAVING** filtra i gruppi dopo l'aggregazione, a differenza di WHERE che filtra le righe prima.

Funzioni su Stringhe

- **UPPER / LOWER / INITCAP** – Conversione maiuscole/minuscole
- **LENGTH** – Lunghezza della stringa
- **SUBSTR(str, pos, len)** – Estrae sottostringa
- **INSTR(str, substr)** – Posizione sottostringa
- **REPLACE(str, old, new)** – Sostituzione testo

Funzioni su Numeri e Date

- **ROUND(n, dec)** – Arrotondamento
- **TRUNC(n, dec)** – Troncamento
- **SYSDATE** – Data/ora corrente del server
- **ADD_MONTHS, MONTHS_BETWEEN** – Calcoli su date

Gestione NULL e Logica Condizionale

NVL(expr, val) sostituisce NULL con un valore default. **COALESCE** restituisce il primo valore non nullo. **CASE** e **DECODE** implementano logica condizionale nelle query.

WHERE vs HAVING

WHERE filtra le righe *prima* dell'aggregazione e non può usare funzioni aggregate. **HAVING** filtra i gruppi *dopo* GROUP BY e può usare COUNT, SUM, AVG, ecc.

```
SELECT department_id,  
       COUNT(*) AS num_dipendenti,  
       AVG(salary) AS media_stipendi,  
       CASE WHEN AVG(salary) > 6000 THEN 'ALTA' ELSE 'NORMALE' END AS fascia  
FROM employees  
GROUP BY department_id  
HAVING AVG(salary) > 3000;
```

JOIN, Subquery, DML, Transazioni e DDL

Il **JOIN** combina righe da più tabelle tramite condizioni di uguaglianza sulle chiavi. Le **subquery** nidificano query all'interno di altre. Il **DML** modifica i dati; le **transazioni** garantiscono atomicità; il **DDL** definisce la struttura del database.

JOIN e Subquery

- **JOIN implicito:** clausola WHERE con condizione di uguaglianza
- **Prodotto cartesiano:** JOIN senza condizione — da evitare
- **Subquery single-row:** restituisce 1 riga, usabile con =
- **Subquery multi-row:** restituisce N righe, usabile con IN
- **EXISTS:** verifica esistenza righe nella subquery

DML e Transazioni

- **INSERT** — Inserisce nuove righe
- **UPDATE** — Modifica valori esistenti
- **DELETE** — Rimuove righe (con WHERE!)
- **COMMIT** — Conferma la transazione
- **ROLLBACK** — Annulla le modifiche
- **SAVEPOINT** — Punto di ripristino parziale

```
-- JOIN tra tre tabelle
SELECT e.first_name, e.last_name, d.department_name, j.job_title
FROM employees e, departments d, jobs j
WHERE e.department_id = d.department_id
      AND e.job_id = j.job_id;

-- DML con transazione
UPDATE employees SET salary = salary * 1.1 WHERE department_id = 50;
COMMIT;
```

DDL — Definizione Struttura

- **CREATE TABLE** — Crea una tabella
- **CREATE VIEW** — Vista virtuale su query
- **CREATE INDEX** — Migliora performance ricerca
- **CREATE SEQUENCE** — Generatore di numeri
- **ALTER** — Modifica struttura esistente
- **DROP** — Elimina oggetto definitivamente
- **TRUNCATE** — Svuota tabella (DDL, non DML!)

DROP vs TRUNCATE vs DELETE: DROP elimina la struttura; TRUNCATE svuota e resetta storage (non rollback); DELETE rimuove righe con WHERE e supporta rollback.

Introduzione a PL/SQL: Blocchi, Variabili e Tipi

PL/SQL è il linguaggio procedurale di Oracle che estende SQL con variabili, strutture di controllo e gestione degli errori. Un **blocco anonimo** è la struttura base: inizia con DECLARE (opzionale), esegue istruzioni in BEGIN e gestisce errori in EXCEPTION.

1

DECLARE

Sezione opzionale per dichiarare variabili, cursori e costanti. Le variabili usano := per l'assegnazione. I tipi includono NUMBER, VARCHAR2, DATE, BOOLEAN.

2

BEGIN

Contiene le istruzioni eseguibili: SQL, assegnazioni, chiamate a DBMS_OUTPUT. Le variabili di tipo %TYPE ereditano il tipo da una colonna, %ROWTYPE da un'intera riga.

3

EXCEPTION

Gestisce errori runtime come NO_DATA_FOUND e TOO_MANY_ROWS. La sezione è opzionale ma fondamentale per blocchi robusti.

4

END /

Chiude il blocco. Lo slash / è il segnale per SQL*Plus e SQL Developer di eseguire il blocco.

```
DECLARE
  v_salary employees.salary%TYPE;
  r_emp     employees%ROWTYPE;
BEGIN
  SELECT * INTO r_emp
  FROM employees
  WHERE employee_id = 101;
  v_salary := r_emp.salary;
  DBMS_OUTPUT.PUT_LINE(r_emp.first_name || ' - ' || v_salary);
END;
/
```

⚠ **Errore Comune:** Dimenticare lo slash / dopo END; impedisce l'esecuzione in SQL*Plus e SQL Developer. Lo slash non è parte del codice PL/SQL, ma il comando di esecuzione per il client.

SQL dentro PL/SQL: SELECT INTO e Cursori Impliciti

PL/SQL permette di eseguire istruzioni SQL direttamente nel blocco. Il comando **SELECT INTO** recupera i valori in variabili, ma deve restituire **esattamente una riga**. Le istruzioni DML (INSERT, UPDATE, DELETE) possono essere usate liberamente. Oracle gestisce automaticamente i **cursori impliciti** per ogni istruzione SQL.

SELECT INTO — Regole Critiche

Deve restituire **una e una sola riga**. Se non trova righe → **NO_DATA_FOUND**. Se trova più righe → **TOO_MANY_ROWS**. Entrambe sono eccezioni gestibili nella sezione EXCEPTION.

DML in PL/SQL

INSERT, UPDATE e DELETE funzionano come in SQL. **COMMIT** e **ROLLBACK** controllano le transazioni. Le funzioni SQL (COUNT, SUM, ecc.) sono utilizzabili nelle espressioni.

Attributi del Cursore Implicito (SQL%)

- **SQL%ROWCOUNT** — Numero di righe interessate dall'ultima istruzione DML
- **SQL%FOUND** — TRUE se l'ultima istruzione ha interessato almeno una riga
- **SQL%NOTFOUND** — TRUE se nessuna riga è stata interessata
- **SQL%ISOPEN** — Sempre FALSE per cursori impliciti (vengono chiusi automaticamente)

Questi attributi sono essenziali per verificare il risultato di operazioni DML e prendere decisioni nel codice.

```
BEGIN
  UPDATE employees
  SET salary = salary * 1.1
  WHERE department_id = 50;

  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Righe aggiornate: ' || SQL%ROWCOUNT);
  ELSE
    DBMS_OUTPUT.PUT_LINE('Nessun dipendente aggiornato');
  END IF;
END;
/
```

Logica di Controllo: IF, CASE e Cicli

PL/SQL offre strutture di controllo complete per implementare logica condizionale e iterativa. La scelta corretta del costrutto migliora leggibilità e manutenibilità del codice.

Strutture Condizionali

IF / THEN / ELSIF / ELSE / END IF valuta condizioni booleane in sequenza. **CASE** è più compatto quando si confronta una variabile con valori fissi. Entrambi supportano nidificazione.

Tipi di Ciclo

- **LOOP / EXIT / EXIT WHEN** — Ciclo base con uscita condizionale
- **WHILE LOOP** — Esegue finché la condizione è TRUE
- **FOR LOOP** — Itera su un intervallo numerico definito
- **CURSOR FOR LOOP** — Itera automaticamente sui risultati di una query

⚠ Errori Comuni nei Cicli

- **Loop infiniti:** dimenticare EXIT WHEN o usare condizione sempre TRUE in WHILE
- **Contatori sbagliati:** nel FOR LOOP il contatore è gestito automaticamente — non modificarlo
- **Off-by-one:** il FOR LOOP include entrambi gli estremi (1..10 = 10 iterazioni)
- **Performance:** evitare cicli che eseguono SELECT INTO per ogni iterazione — preferire cursori bulk

📌 **Regola:** Usare FOR LOOP quando si conosce il numero di iterazioni; WHILE LOOP quando la condizione dipende da calcoli runtime; CURSOR FOR LOOP per elaborare risultati di query.

```
DECLARE
  v_salary employees.salary%TYPE;
  v_level VARCHAR2(20);
BEGIN
  SELECT salary INTO v_salary
  FROM employees
  WHERE employee_id = 101;

  IF v_salary < 3000 THEN
    v_level := 'LOW';
  ELSIF v_salary < 6000 THEN
    v_level := 'MEDIUM';
  ELSE
    v_level := 'HIGH';
  END IF;

  DBMS_OUTPUT.PUT_LINE(v_level);
END;
/
```

Gestione degli Errori: Sezione EXCEPTION

La sezione **EXCEPTION** è il cuore della robustezza in PL/SQL. Permette di intercettare e gestire errori runtime senza interrompere l'esecuzione in modo incontrollato. Oracle distingue tra **eccezioni predefinite** (NO_DATA_FOUND, TOO_MANY_ROWS) ed **eccezioni utente** definite con DECLARE e RAISE.

Eccezioni Principali

- **NO_DATA_FOUND** — SELECT INTO non restituisce righe
- **TOO_MANY_ROWS** — SELECT INTO restituisce più di una riga
- **VALUE_ERROR** — Errore di conversione o overflow
- **ZERO_DIVIDE** — Divisione per zero
- **WHEN OTHERS** — Cattura tutte le eccezioni non gestite

SQLCODE e SQLERRM

SQLCODE restituisce il codice numerico dell'errore; **SQLERRM** restituisce il messaggio descrittivo. Utili in WHEN OTHERS per diagnosticare errori imprevisti.

Best Practice nella Gestione Errori

- Gestire sempre le eccezioni **specifiche prima** di WHEN OTHERS
- **Mai** usare WHEN OTHERS THEN NULL — nasconde gli errori rendendo il debug impossibile
- In WHEN OTHERS, usare sempre **RAISE** o **RAISE_APPLICATION_ERROR** per propagare l'errore
- Loggare SQLCODE e SQLERRM per tracciare gli errori in produzione
- Testare i percorsi di errore con dati che li generano intenzionalmente

```
DECLARE
  v_salary employees.salary%TYPE;
BEGIN
  SELECT salary INTO v_salary
  FROM employees
  WHERE employee_id = -1; -- ID inesistente

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Dipendente non trovato');
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Troppi risultati restituiti');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE(SQLCODE || ' - ' || SQLERRM);
    RAISE; -- Propaga l'errore al chiamante
END;
/
```


Cursori Espliciti e Cursor FOR LOOP

I cursori espliciti sono necessari quando SELECT INTO non è sufficiente — ovvero quando la query restituisce **più righe**. Un cursore esplicito offre controllo completo sul ciclo di vita dell'esecuzione della query. Il **Cursor FOR LOOP** semplifica drasticamente il codice automatizzando OPEN, FETCH e CLOSE.

Ciclo di Vita del Cursore Esplicito

1. **DECLARE CURSOR** — Definisce la query associata al cursore
2. **OPEN** — Esegue la query e popola il result set
3. **FETCH** — Recupera una riga alla volta nelle variabili
4. **CLOSE** — Rilascia le risorse associate al cursore

Attributi del Cursore Esplicito

- **%FOUND** — TRUE se l'ultimo FETCH ha restituito una riga
- **%NOTFOUND** — TRUE se l'ultimo FETCH non ha restituito righe
- **%ROWCOUNT** — Numero di righe recuperate finora
- **%ISOPEN** — TRUE se il cursore è attualmente aperto

Perché Usare il Cursor FOR LOOP

Il Cursor FOR LOOP è la forma **raccomandata** nella maggior parte dei casi. Elimina la necessità di dichiarare variabili per il FETCH, gestisce automaticamente OPEN e CLOSE, e riduce il rischio di errori (cursore dimenticato aperto).

- Nessun DECLARE CURSOR esplicito necessario
- La variabile del loop è di tipo %ROWTYPE automaticamente
- Il cursore viene chiuso automaticamente al termine
- Codice più compatto, leggibile e manutenibile

ⓘ Quando usare cursori espliciti: Solo quando serve controllo fine (FETCH con LIMIT, riapertura del cursore, passaggio del cursore come parametro).

```
-- Cursor FOR LOOP (forma raccomandata)
BEGIN
  FOR r_emp IN (
    SELECT employee_id, first_name, salary
    FROM employees
    WHERE department_id = 50
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      r_emp.employee_id || ' - ' ||
      r_emp.first_name || ' - ' ||
      r_emp.salary
    );
  END LOOP;
END;
/
```

Procedure, Function, Package, Trigger e Caso Finale

Gli oggetti PL/SQL memorizzati nel database permettono di organizzare, riutilizzare e mantenere il codice in modo professionale. Le **procedure** eseguono azioni; le **function** restituiscono valori; i **package** raggruppano oggetti correlati; i **trigger** reagiscono automaticamente a eventi DML.

Procedure vs Function

- **PROCEDURE:** esegue azioni, modifica dati, può avere parametri OUT. Non può essere usata in SELECT.
- **FUNCTION:** restituisce un valore con RETURN. Può essere usata in query SQL.
- **Parametri:** IN (input), OUT (output), IN OUT (entrambi)
- **CREATE OR REPLACE:** ricrea l'oggetto senza errori se esiste già

Package e Trigger

- **PACKAGE SPEC:** dichiara interfacce pubbliche (procedure, function, variabili)
- **PACKAGE BODY:** implementa la logica, può contenere elementi privati
- **Trigger BEFORE/AFTER:** si attiva prima o dopo INSERT, UPDATE, DELETE

Quando Usare Cosa

- **Function:** calcoli riutilizzabili in query (es. get_bonus)
- **Procedure:** operazioni DML complesse, batch, elaborazioni
- **Package:** raggruppare logica correlata, nascondere dettagli, gestire stato
- **Trigger:** validazioni automatiche, audit trail, integrità referenziale avanzata



Conclusione: SQL legge e manipola i dati. PL/SQL aggiunge logica, controllo del flusso e riutilizzo. Insieme formano la base dello sviluppo professionale su Oracle Database.

```
-- Function in Package
CREATE OR REPLACE PACKAGE pkg_emp AS
    FUNCTION get_bonus(p_salary NUMBER) RETURN NUMBER;
END pkg_emp;
/

CREATE OR REPLACE PACKAGE BODY pkg_emp AS
    FUNCTION get_bonus(p_salary NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN p_salary * 0.1;
    END;
END pkg_emp;
/

-- Caso finale: cursore + function + update
BEGIN
    FOR r_emp IN (
        SELECT employee_id, salary
        FROM employees
        WHERE department_id = 50
    ) LOOP
        UPDATE employees
        SET salary = salary + pkg_emp.get_bonus(r_emp.salary)
        WHERE employee_id = r_emp.employee_id;
    END LOOP;
    COMMIT;
END;
/
```